File: `src/hooks/useChartPanels.js`

Category: Custom React Hook (Chart Panel State Manager)
Runtime: Browser (React)
Primary consumer: `src/components/ShellyDashboard.jsx`
Direct UI consumer: `src/components/ChartPanel.jsx` (via the returned actions/selectors)
Upstream dependencies: `src/config/seriesCatalog.js` (`CHART_KEYS`, `SERIES_CATALOG`)

Overview

`useChartPanels` manages the **entire state model for the dynamic multi-chart system**.

It provides:

- A list of chart panels (`charts`)
- Functions to add/remove charts
- Functions to add/remove metric series within a chart
- Functions to edit per-series style (color, curvature, thickness)
- Functions to manage Y-axis auto-range reporting and Y-axis locking
- Helper "selector" functions used by UI to build dropdown choices and chart series definitions

This hook is designed as a **state manager**. It does not render UI and does not compute chart values—those are handled by components (especially `LineGraph.jsx`).

Imports

- `useMemo`, `useState` from React
- `CHART_KEYS`, `SERIES_CATALOG` from `../config/seriesCatalog`

Data Structures (Internal Types)

Chart panel state (`ChartPanelState`)

Each chart entry inside `charts` has this shape:

```
ChartPanelState:
  id: string
```

```
  selectedSeries: SeriesConfig[]
  pendingAddKey: string

  currentYMin: number | null
  currentYMax: number | null

  lockYAxis: boolean
  lockedYMin: number | null
  lockedYMax: number | null
```

Series configuration (`SeriesConfig`)

Each selected metric inside a chart has:

```
SeriesConfig:
  key: string
  color: string
  tension: number
  thickness: number
```

Internal Helper Functions

`createChartPanel()`

Signature

`createChartPanel() → ChartPanelState`

Purpose

Creates a new, empty chart panel object with defaults.

Default values

- `selectedSeries`: empty array (no metrics selected yet)
- `pendingAddKey`: first available chartable key (`CHART_KEYS[0]`) or `""`
- Y-axis:
  - `currentYMin/currentYMax`: null until chart computes auto-range
  - `lockYAxis`: false
  - `lockedYMin/lockedYMax`: null

ID generation

- Uses `crypto.randomUUID()` if available
- Falls back to a time+random string

`computeAddableKeys(panel)`

Signature

```
computeAddableKeys(panel: ChartPanelState) → string[]
```

Purpose

Returns the list of metric keys that can still be added to a chart panel.

Rules

- A metric key is "addable" if:
  - It is in `CHART_KEYS`, and
  - It is not already present in `panel.selectedSeries`

This prevents duplicate series in the same chart.

Exported Hook API

`useChartPanels(options?)`

Signature

```
useChartPanels(options?: { startEmpty?: boolean, insertAtTop?: boolean })
  → {
      charts: ChartPanelState[],

      addChart(): void,
      removeChart(chartId: string): void,

      setPendingAddKey(chartId: string, key: string): void,
      addSeriesToChart(chartId: string): void,
      removeSeriesFromChart(chartId: string, key: string): void,

      setSeriesColor(chartId: string, key: string, color: string): void,
      setSeriesTension(chartId: string, key: string, tension: number): void,
      setSeriesThickness(chartId: string, key: string, thickness: number):
void,

      setChartAutoYRange(chartId: string, yMin: number, yMax: number): void,
      toggleLockYAxis(chartId: string, yMin?: number, yMax?: number): void,

      getAddableKeys(chart: ChartPanelState): string[],
      buildChartSeries(chart: ChartPanelState): ChartSeries[],
      getSafePending(chart: ChartPanelState): string
    }
```

Options

- `startEmpty` (default `true`)
  - If true: begin with zero charts.
  - If false: begin with one chart already created.
- `insertAtTop` (default `true`)
  - If true: new charts appear at the top of the list.
  - If false: new charts append at the end.

Internal State

`charts`

```
const [charts, setCharts] = useState(() => (startEmpty ? [] :
[createChartPanel()]));
```

- Holds all chart panels.
- Updating it triggers re-render of the dashboard and all chart panels.

Action Functions (State Transitions)

All action functions update state using immutable patterns (`map`, `filter`) to remain compatible with React state update expectations.

`addChart()`

Signature

```
addChart() → void
```

Behavior

- Creates a new chart panel object via `createChartPanel()`
- Inserts it:
  - at the top if `insertAtTop` is true
  - at the bottom otherwise

Side effects

- React state update only

`removeChart(chartId)`

Signature

```
removeChart(chartId: string) → void
```

Behavior

- Removes the chart matching `chartId` using `filter`.

`setPendingAddKey(chartId, key)`

Signature

```
setPendingAddKey(chartId: string, key: string) → void
```

Behavior

- Updates the dropdown selection for that chart.

This does not add a series—it only changes what will be added when the user clicks "+ Add".

`addSeriesToChart(chartId)`

Signature

```
addSeriesToChart(chartId: string) → void
```

Behavior (step-by-step)

1. Computes addable keys for this chart (metrics not already selected).
2. Chooses which key to add:
   - If `pendingAddKey` is valid → use it
   - Otherwise fall back to the first addable key
3. Looks up metric definition in `SERIES_CATALOG`
4. Appends a new `SeriesConfig` entry using defaults:
   - `color`: metric defaultColor
   - `tension`: 0.05
   - `thickness`: 2
5. Updates `pendingAddKey` to the next addable key (or `""` if none remain)

Why `pendingAddKey` is updated

So the dropdown always points at a valid remaining metric after adding.

`removeSeriesFromChart(chartId, key)`

Signature

```
removeSeriesFromChart(chartId: string, key: string) → void
```

Behavior

- Removes the series from `selectedSeries`
- Recomputes addable keys and adjusts dropdown selection accordingly

Special case
If the last series is removed (chart becomes empty):

- Resets auto-range state:
  - `currentYMin/currentYMax = null`
- Resets Y-axis lock state:
  - `lockYAxis = false`
  - `lockedYMin/lockedYMax = null`

This prevents a “locked axis” remaining active when there is no data.

`setSeriesColor(chartId, key, color)`

Signature

```
setSeriesColor(chartId: string, key: string, color: string) → void
```

Updates the color of the specified metric line.

`setSeriesTension(chartId, key, tension)`

Signature

```
setSeriesTension(chartId: string, key: string, tension: number) → void
```

Updates the curvature/smoothing setting of the line.

`setSeriesThickness(chartId, key, thickness)`

Signature

```
setSeriesThickness(chartId: string, key: string, thickness: number) → void
```

Updates the line width.

Y-axis Management

`setChartAutoYRange(chartId, yMin, yMax)`

Signature

```
setChartAutoYRange(chartId: string, yMin: number, yMax: number) → void
```

Purpose
Stores the most recent automatically calculated Y-range reported by `LineGraph`.

Important guard
If `currentYMin/currentYMax` are already equal to the incoming values, the chart state is not updated.

This prevents unnecessary re-renders and avoids potential update loops.

`toggleLockYAxis(chartId, yMin?, yMax?)`

Signature

```
toggleLockYAxis(chartId: string, yMin?: number, yMax?: number) → void
```

Behavior

- If currently locked:
  - Unlocks and clears locked values
- If currently unlocked:
  - Locks using:
    - provided `yMin/yMax`, OR
    - the latest stored `currentYMin/currentYMax`

If `yMin/yMax` are not valid numbers, it does not lock.

Selector / Helper Functions (Returned to UI)

These do not modify state. They support UI rendering.

`getAddableKeys(chart)`

Signature

```
getAddableKeys(chart: ChartPanelState) → string[]
```

Returns addable metrics (not currently selected).

`getSafePending(chart)`

Signature

```
getSafePending(chart: ChartPanelState) → string
```

Ensures dropdown value is valid:

- returns current `pendingAddKey` if still addable
- otherwise returns first addable key or empty string

`buildChartSeries(chart)`

Signature

```
buildChartSeries(chart: ChartPanelState) → ChartSeries[]
```

Builds objects for `LineGraph` to render.

Each returned series includes:

- label
- unit
- style (color, tension, thickness)
- valueAccessor from SERIES_CATALOG

This is the bridge between “what the user selected” and “what the chart renderer needs”.

Hook memoization (`useMemo`)

Actions object memoization

```
useMemo(() => { ...return actions... }, [insertAtTop])
```

Why `insertAtTop` is included

Because `addChart()` behavior depends on it. If the option changes, the action functions must be regenerated.

Selector object memoization

```
useMemo(() => { ...return selectors... }, [])
```

Selectors depend only on imported constants and the input argument you pass them. They do not depend on changing state captured in closures, so they can be created once.

Side effects

- No external side effects (no network, no timers)
- Only React state updates